Competition day: bot operator quickstart (CTC 2026, Systematic Equities)

Check-in 8:00-8:30 (confirm the date and the live host on Slack #questions). Keep this page, `ANNOUNCEMENT_CARD.md` and `RUNBOOK.md` (the detail behind every step) next to the laptop. **Copy commands from the `.md` file, not from the PDF.** Every path is absolute and quoted, because the folder name `Fun Stuff` has a space.

Fill in	
Live host, confirmed by the organizers on Slack #questions	_____ (testnet: `cornelltradingcompetition.org`)
Config hash and phrase from C2 or C4	`LIVE` _____ _____
Second-network address (phone over USB), if used	_____
Operator (console) / watcher (alerts) / analyst (announcements) / browser (cancel-all, logged in)	_____ / _____ / _____ / _____

The commands use `cornelltradingcompetition.org`. **If the organizers name another host, replace it in C2, C3, C4 and C5.** Run directories: `runs/comp_dry` (dry run) and `runs/comp_live` (live).

1. Night before and morning

1. **Power.** Laptop on the AC adapter, lid open, Focus/Do Not Disturb on, no OS update pending. Keep the Mac awake for the whole session in a spare terminal tab (Ctrl-C ends it): `caffeinate -dims`
2. **Stop the overnight soak** before C2 (it stops by itself at 8:30 am, and it holds 3 of the team's 25 market-data connections). Stop its supervisor first, or it restarts the bot within a minute: `pkill -f "overnight_20261024/overnight.sh"`, then `pkill -INT -f "runs/overnight_2026"` and `pkill -INT -f "record_all.py"`, and the offline research runs with `pkill -f "sim/run_research.py"`. Check that `pgrep -fl "overnight.sh|run_bot.py|record_all.py|run_research.py"` prints nothing.
3. **Two networks (optional).** The config supports it: `feed.connections.1.local_addr` binds the second market-data connection (`ws1`) to a local address; `ws0` and every REST request (orders included) use the Mac's default route. A Mac joins one Wi-Fi network at a time, so the second path is the phone **over a USB cable** (iPhone: Personal Hotspot on, plug in; Android: USB tethering).
   - `networksetup -listallhardwareports` gives the "iPhone USB" device (for example `en7`), then `ipconfig getifaddr en7` its address (iPhone hotspots give `172.20.10.x`).
   - Keep Wi-Fi above the phone in System Settings > Network > ... > Set Service Order, so orders go over Wi-Fi.
   - Add `--set 'feed.connections.1.local_addr=["172.20.10.2", 0]'` (your address) to **C2, C3 and C5**; it changes the config hash and so the live phrase.
   - Not verifiable offline: the C2 line `market data ws1 (local_addr ...)` must PASS and the dry run show `ws 2/2`. If the phone's address changes, `ws1` cannot bind (`ws 1/2`) and trading continues on `ws0`; fix it between rounds with a restart. No second network: skip this step.
4. **The key** may be new on the day. Never paste it into Slack, a chat, an email, a command line or a project file. Copy only the key to the clipboard from the organizers' message, then run these lines one by one:

```
umask 077
pbpaste | tr -d '[:space:]' > "/Users/bowentian/.ctc_bot_key.new"
chmod 600 "/Users/bowentian/.ctc_bot_key.new"
mv "/Users/bowentian/.ctc_bot_key" "/Users/bowentian/.ctc_bot_key.old"
mv "/Users/bowentian/.ctc_bot_key.new" "/Users/bowentian/.ctc_bot_key"
pbcopy < /dev/null
```

Then run C2: it checks the format and the mode (600) without printing the key, and the exchange's answer shows whether the key works. If it is rejected (HTTP 401/403), put the old one back with `mv "/Users/bowentian/.ctc_bot_key.old" "/Users/bowentian/.ctc_bot_key"`. Never `cat` the key file.

2. Checks before going live (C1, C2, C3 in order)

C1. Offline self-test (2 s, sends nothing). Expect exit 0 and six `[ok]` lines: config, research case, engine snapshot-fix probe, cost model, settlement laws, case diff.

```
"/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/.venv/bin/python" -I \
  "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/run_bot.py" --self-test
```

C2. Preflight (about 30 s; GET requests and two market-data connections only, never an order action).

```
"/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/.venv/bin/python" -I \
  "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/preflight.py" \
  --host cornelltradingcompetition.org --key-file /Users/bowentian/.ctc_bot_key
```

It ends with `RESULT: PASS`, `WARN` or `FAIL` (exit 1). Fix every FAIL; read every WARN with the team.

- `key file` PASS (mode 0600). `config` prints the hash and the phrase a `--live` run with the same flags asks for.
- `competition time`: state (`running` in a round, `ready` between rounds), competition id, REST latency.
- `case.json`: "identical to the research case.json, timing included". A `CASE TIMING DIFFERS` banner means the windows move with the served timing but the strategies were calibrated on the research timing: ask the organizers, decide as a team, and only then use `--accept-timing-diff` in C5. Symbol differences: the analyst checks them; S1-r3 skips those symbols until `caseok SYM`.
- `positions and open orders`: flat and none. Anything else: find out who trades on the key before going live.
- `market data ws0` / `ws1`: frames by type. "no market data" between rounds is normal; a FAIL is not.

C3. Dry run on the live host, 2 minutes (sends no order action without `--live`).

```
"/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/.venv/bin/python" -I \
  "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/run_bot.py" \
  --host cornelltradingcompetition.org --key-file /Users/bowentian/.ctc_bot_key \
  --run-dir "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/runs/comp_dry"
```

Check, in order:

- the banner ends `(DRY-RUN)`, then `running (DRY-RUN); type 'help' ...`; the status line (every 2 s) starts `[DRY]`;
- `ws 2/2` with a frame age under about 3 s while a round runs (both connections up);
- the clock: `r<round> <phase> t=<seconds> server` (or `feed`) and `t` moving; not `?` / `unknown` in a round;
- `eng` age of a few seconds at most in a round, and no `[ALERT] engine_snapshot_bug` (the engine probe passed);
- no `[ALERT] auth_failed`, `ws_rejected`, `case_fetch_failed` or `case_diff` (unless already reviewed in C2);
- no `note: N open orders exist on this key` line;
- kill drill: type `k` (expect `KILLED: ...`), then `ord`, then `unfreeze-kill`; finally `q` (exits 0).

Start the live bot before round 1 opens. A bot that joins mid-season has missed the earlier rounds' storms, so in round 3 S1-r3 skips the symbols it cannot value or check (the 23 Oct testnet dry run that joined mid-season traded only REIN in that round 3). After a late start or `--fresh`, the analyst decides `lawok SYM` or `lawok all` (it clears only the rounds 1-2 law-check gate).

3. Going live (C4, C5)

Rule: go live only after the team has seen C3 work on the live host, with the same host and flags. As of 24 Oct the bot had never sent a live order (STATUS.md), so these may be its first. A cautious start is `--set sizing.scale=0.25` (team decision; it changes the hash). Stop the C3 dry run with `q` first: one bot per key.

C4. Print the config hash (offline). For the default config on `cornelltradingcompetition.org` it is `70d9aac5c403`, so the phrase is `LIVE cornelltradingcompetition.org 70d9aa`. Any `--set` / `--profile` /other host changes it; a hash you did not expect means a flag you did not agree.

```
"/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/.venv/bin/python" -I \
  "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/run_bot.py" \
  --print-config --host cornelltradingcompetition.org | grep '^#'
```

C5. Live (add the same `--set` flags as in C3, and `--accept-timing-diff` only after a team decision):

```
"/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/.venv/bin/python" -I \
  "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/run_bot.py" --live \
  --host cornelltradingcompetition.org --key-file /Users/bowentian/.ctc_bot_key \
  --run-dir "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/runs/comp_live"
```

The prompt shows `LIVE TRADING: real orders will be sent`, the host, the config hash, the profiles, the config differences from default.json (expect only the four `exchange.*` host lines plus your agreed `--set` s), the case.json differences (expect `(none)`), the enabled strategies (`announce, flatten, h1, rein_ask, s1r3, s3`), `sizing.scale`, the action cap (`150 per 61.0 s`) and any warning (open orders on the key, laws module missing). **Read it all.** Then `Type exactly: LIVE <host> <first 6 characters of the config hash>` and `confirmation (1/3):`. Type `LIVE`, a space, the host exactly as printed (lower case, no port), a space, the six hash characters, then Enter. Three wrong tries or Ctrl-D exit with code 2 and send nothing. Then `[ALERT] live: LIVE trading confirmed ...`, `running (LIVE)` and a status line starting `[LIVE]`. `--live` is refused (exit 2, nothing sent) without a terminal, with a `KILL` file in the run dir, without a served case.json, on a timing difference without `--accept-timing-diff`, after a failed engine probe, or with a config outside the live bounds.

4. During the rounds

Round clock (research timing, real seconds from the open): H1 entry 50-56 s, storm 60 s, H1 check 58-90 s, S3 from 80 s, storm end 540 s, rounds 1-2 flatten 540-568 s, preliminary report 570 s, **final report 740 s**, close 780 s. H1 now exits by itself (84-98 s) in a round where S3 is off.

Status line: `[LIVE] NORMAL e0 r2 storm t=213.4 server | ws 2/2 0.4s wm695 gaps0 dup0 | eng 0.3s 41ms q0 | act 87/150 rej0 unk0 | mtm +3,210 pk +4,020 | PORT+200 HAVEN-120 | frz - | holds -`. Watch: the mode (`NORMAL`; `NO_NEW_RISK` means a hold, `FROZEN` a kill), `ws 2/2` and frame age, `t` moving, `eng` age under 2 s in the storm (S3 skips older values), `act` below 150 with `rej0 unk0`, `mtm` against `pk` (new risk stops $20k below the round's peak), `frz` (announcement freezes) and `holds`.

Watcher terminal (alerts and S1-r3 decisions as they happen):

```
tail -F "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/runs/comp_live/events.jsonl" | grep --
line-buffered -E '"kind":"(alert|s1r3_fire|s1r3_gate)"'
```

Console (type, then Enter)	Effect
`k`	**Kill**: freeze, cancel all, follow-ups until nothing is open. Positions are not flattened. `unfreeze-kill` (in full) releases it
`p` / `r`	Pause new risk (reducing orders still go) / resume (releases the pause, drawdown, restart, feed_down, auth, season and comp-id holds)

Console (type, then Enter)	Effect
`x SYM` / `e SYM`	Disable a symbol (its orders cancelled, reducing only) / enable it
`s3 off`, `s3 on`, `s3 rounds 3`	S3 switch; with S3 off for a round, H1 exits by itself at 84-98 s
`h1 off`, `h1 on`, `h1 0.5`, `h1 auto`	H1 switch; a size 0-1 overrides the automatic size until `h1 auto`
`ua SYM`, `u SYM`, `u all`, `ov PARAM X`, `ov PARAM off`	Announcements: follow **ANNOUNCEMENT_CARD.md** (`ov` first, then `ua SYM`; `u SYM` for unaffected names; `u all` unfreezes every frozen name after a no-effect headline)
`s1ok SYM`, `x1 SYM`, `lawok SYM` or `lawok all`, `caseok SYM` / `all` / `timing`	S1-r3 and case gates, as the analyst decides
`st`, `pos`, `ord`, `holds`, `ann`, `help`	Show status, positions, live orders, holds, announcements, all commands
`b 120`	Lower the bot's action cap (maximum 150)
`q`	Quit: kill, wait until every order is cancelled, exit 0

Round 3, final report (740 s). From 680 s the bot keeps the action budget free; at 740 s S1-r3 takes at settlement value ± ε within about 0.3 s, rests the other side, sweeps until 778 s, and cancels the REIN ask. Hands off the keyboard from 735 to 760 s unless something is clearly wrong: a `k` cancels S1-r3's orders. After about 700 s an announced symbol enters S1-r3 only after `s1ok SYM`, confirmed by a second person. Before 740 s the analyst reviews the rounds 1-2 `law_check_failed` alerts (S1-r3 skips those symbols) and decides `lawok SYM`. The watcher reads the `s1r3_gate` lines; the analyst decides `x1 SYM`. Round-3 positions settle at the close; they are not flattened.

Disconnects. `ws 1/2`: one connection is down and reconnects every 0.5 s; nothing to do. `ws 0/2` for 10 s: `[ALERT] feed_down`, the bot cancels everything and holds new risk; it resumes by itself after a snapshot (`r` forces it once data flows again). Switch networks if the venue Wi-Fi is down. `ws_rejected` 429: too many connections (25 per team: close extra bots, recorders, preflights); 401/403: the key. `api_errors` or `reconcile_down`: REST is failing and the bot cannot see its fills; it releases by itself, and if orders may be live while the exchange is unreachable, cancel all in the browser.

HTTP 429 on REST. Automatic: the bot holds new orders for 5 s (`http_429`) and pads its action ledger; the kill still works. Humans stop all manual browser trading (it counts against the team's 180 actions per minute). If it repeats, `b 120`.

Crash or frozen process.

1. If it was live: **cancel all in the browser first.**
2. Restart with the exact C5 command (same `--run-dir`, same flags, so the same phrase). The state is saved (`state.json`, within 1 s of every order change); with the same competition id it prints `saved state: restore ...`, restores positions, orders and the engine, and holds new risk (`[ALERT] restart`).
3. Check `pos` and `ord` against the browser, then `r`.
4. A `KILL` file in the run dir blocks the start: remove it once you know why it is there: `rm "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/runs/comp_live/KILL"`.
5. `--fresh` only if `state.json` is corrupt (the engine then starts empty; see the lawok note in section 2).

5. Emergency: stop everything

1. **`k`** then Enter in the bot console.
2. From any other terminal (works within 0.25 s): `touch "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/runs/comp_live/KILL"` (remove it before any restart, section 4).
3. **Ctrl-C** in the bot terminal: kill, then exit within 5 s (code 130). A second Ctrl-C sends one immediate cancel-all and exits at once.
4. **Browser cancel-all** on the exchange's web page, by the person logged in. Always, if the bot is dead or unreachable while live.

None of these flattens positions. Afterwards `ord` (the bot's view) and the browser (the exchange's view) must both show no open order. Any doubt: `k` first, questions afterwards.

6. After each round and at the end

- **After rounds 1 and 2:** flat except REIN by 568 s (`pos`, browser). On `not_flat` or `flatten_behind` the team decides whether to flatten by hand in the browser. `holds` should be `-`; read the watcher's alerts.
- **After round 1:** H1's size for rounds 2-3 follows the round-1 check (`h1_check` pass: full size; fail: half). Override with `h1 X` only on a team decision.
- **Between rounds:** no code changes. A config change means `q`, then C5 with the new flag (new hash, new phrase; the saved state is restored), then `pos`, `ord` and `r`.
- **At the end:** `q` and wait for the exit (code 0). Check in the browser that nothing is open and note the final positions. Keep both run directories and copy them off the laptop: `ditto -c -k --keepParent "/Users/bowentian/Documents/Fun Stuff/CTC2026/systematic/bot/runs/comp_live" "/Users/bowentian/Desktop/comp_live.zip"`